

Módulo 3 – Model Development Support (12h)

3.1 Objetivos

Ao final deste módulo, os formandos deverão ser capazes de:

- Compreender o papel do desenvolvimento de modelos no ciclo de vida de um projeto de Ciência de Dados. Exemplo prático: Uma empresa de logística desenvolve um modelo para prever atrasos nas entregas com base em dados históricos e meteorológicos.

Nota teórica: O ciclo de vida de um projeto de Ciência de Dados inclui etapas como recolha, preparação, modelação, validação e implementação. Cada fase influencia a qualidade final do modelo, sendo essencial compreender como as decisões iniciais afetam o desempenho preditivo.

- Apoiar na criação de modelos preditivos supervisionados e não supervisionados.

Nota teórica: Modelos supervisionados utilizam dados rotulados para aprender relações entre variáveis (ex.: regressão, classificação), enquanto modelos não supervisionados identificam padrões sem rótulos (ex.: clustering, PCA). A escolha depende do tipo de problema e da disponibilidade de rótulos.

- Realizar engenharia de variáveis (*feature engineering*) para melhorar a performance dos modelos.

Nota teórica: *Feature engineering* consiste em transformar, combinar ou criar variáveis relevantes para aumentar a capacidade preditiva do modelo. Esta etapa é frequentemente mais determinante para o desempenho do que a escolha do algoritmo.

- Selecionar e aplicar algoritmos adequados ao tipo de problema.

Nota teórica: A seleção de algoritmos depende da natureza dos dados, do objetivo (regressão ou classificação), da interpretabilidade desejada e da complexidade computacional. Algoritmos simples podem superar modelos complexos quando os dados são bem preparados.

- Avaliar modelos utilizando métricas apropriadas e técnicas de validação.

Nota teórica: Métricas como RMSE, MAE, Acurácia ou F1-score permitem quantificar o desempenho do modelo. A validação cruzada reduz o risco de sobreajuste ao testar o modelo em múltiplas partições dos dados.

- Documentar e comunicar o processo de modelagem e os resultados obtidos.

Nota teórica: A documentação garante reprodutibilidade e transparência. A comunicação clara dos resultados facilita a tomada de decisão e aumenta a confiança dos stakeholders no modelo.

3.2 Conteúdos Desenvolvidos

- Introdução ao fluxo de trabalho de modelagem preditiva — preparação de dados, treino, validação, teste e implementação.

Nota teórica: O fluxo de trabalho preditivo segue uma sequência lógica que assegura que o modelo generaliza bem para novos dados. A separação entre treino, validação e teste evita estimativas enviesadas de desempenho.

- Preparação dos dados: divisão treino/validação/teste; normalização e padronização; codificação de variáveis categóricas.

Nota teórica: Normalização e padronização ajustam escalas numéricas, evitando que variáveis com maior magnitude dominem o modelo. A codificação transforma categorias em representações numéricas adequadas para algoritmos matemáticos.

- Engenharia de variáveis: criação de novas variáveis; seleção de variáveis relevantes; redução de dimensionalidade (PCA).

Nota teórica: A redução de dimensionalidade, como o PCA, permite condensar informação em menos variáveis, reduzindo ruído e melhorando a eficiência computacional, especialmente em datasets de alta dimensão.

- Treino de modelos: uso de `caret`, `tidymodels` e `recipes`; algoritmos comuns (regressão linear/logística, árvores, random forest, k-NN).

Nota teórica: Cada algoritmo possui pressupostos e características distintas. Por exemplo, regressão linear assume relações lineares, enquanto random forest combina múltiplas árvores para reduzir variância e melhorar robustez.

- Avaliação de modelos: métricas para regressão (RMSE, MAE, R^2) e classificação (Acurácia, Precisão, Recall, F1-score, AUC); validação cruzada.

Nota teórica: Métricas diferentes capturam aspectos distintos do desempenho. Por exemplo, o F1-score é útil em datasets desbalanceados, enquanto o AUC mede a capacidade discriminativa do modelo.

- Comparação e seleção de modelos.

Nota teórica: A comparação sistemática entre modelos permite identificar o melhor compromisso entre desempenho, interpretabilidade e custo computacional. Técnicas como *grid search* e *random search* ajudam na otimização de hiperparâmetros.

3.3 Atividades Práticas Detalhadas

- Criar um modelo de regressão para prever preços de imóveis utilizando `caret`.

Nota teórica: A regressão é adequada para prever variáveis contínuas. O uso de `caret` simplifica o processo ao fornecer uma interface unificada para treino, validação e tuning.

- Desenvolver um modelo de classificação para prever *churn* de clientes com `tidymodels`.

Nota teórica: Problemas de *churn* são típicos de classificação binária. A escolha de métricas como Recall ou AUC é crucial quando o custo de perder clientes é elevado.

- Implementar um *recipe* para normalizar dados, criar variáveis derivadas e codificar variáveis categóricas.

Nota teórica: O uso de `recipes` permite encapsular todo o pré-processamento num pipeline reprodutível, garantindo consistência entre treino e teste.

- Comparar o desempenho de pelo menos dois algoritmos diferentes para o mesmo problema.

Nota teórica: Comparações controladas permitem avaliar se melhorias são devidas ao algoritmo ou ao pré-processamento. A validação cruzada é essencial para conclusões fiáveis.

- Documentar o processo de modelagem e apresentar os resultados com métricas e gráficos.

Nota teórica: A visualização de resultados facilita a interpretação de padrões, erros e desempenho relativo entre modelos.

3.4 Recursos e Ferramentas

- Software: R e RStudio.

Nota teórica: R é amplamente utilizado em estatística e modelação, oferecendo uma vasta coleção de pacotes especializados. RStudio fornece um ambiente integrado que facilita o desenvolvimento.

- Pacotes: `caret`, `tidymodels`, `recipes`, `ggplot2`, `dplyr`.

Nota teórica: Estes pacotes seguem princípios de coerência e modularidade, permitindo construir pipelines robustos e reprodutíveis.

- Datasets: preços de imóveis, churn de clientes, datasets públicos do UCI Machine Learning Repository.

Nota teórica: O uso de datasets reais ou amplamente estudados permite comparar resultados com benchmarks e compreender limitações práticas.

3.5 Estudo de Caso – Desenvolvimento de Modelo em R

Objetivo: Demonstrar todas as etapas do desenvolvimento de um modelo preditivo.

Nota teórica: Estudos de caso integram teoria e prática, permitindo observar como decisões de pré-processamento, seleção de algoritmos e avaliação influenciam o desempenho final.

```
1 library(tidyverse)
2 library(caret)
3 library(tidymodels)
4
5 # 1. Importar dataset
6 dados <- read_csv("precos_imoveis.csv")
7
8 # 2. Divisão treino / teste
9 set.seed(123)
10 divisao <- initial_split(dados, prop = 0.7)
11 treino <- training(divisao)
12 teste <- testing(divisao)
13
14 # 3. Recipe de pré-processamento
15 receita <- recipe(preco ~ ., data = treino) %>%
16   step_normalize(all_numeric_predictors()) %>%
17   step_dummy(all_nominal_predictors())
18
19 # 4. Modelo regressão linear
20 modelo_rl <- linear_reg() %>% set_engine("lm")
21
22 # 5. Workflow
23 workflow_rl <- workflow() %>%
24   add_recipe(receita) %>%
25   add_model(modelo_rl)
26
27 # 6. Treinar
28 modelo_treinado <- fit(workflow_rl, data = treino)
29
30 # 7. Avaliar
```

```

31 predicoes <- predict(modelo_treinado, teste) %>%
32 bind_cols(teste)
33 metrics(predicoes, truth = preco, estimate = .pred)
34
35 # 8. Comparar com Random Forest
36 modelo_rf <- rand_forest() %>%
37 set_engine("ranger") %>%
38 set_mode("regression")
39
40 workflow_rf <- workflow() %>%
41 add_recipe(receita) %>%
42 add_model(modelo_rf)
43
44 modelo_rf_treinado <- fit(workflow_rf, data = treino)
45
46 predicoes_rf <- predict(modelo_rf_treinado, teste) %>%
47 bind_cols(teste)
48 metrics(predicoes_rf, truth = preco, estimate = .pred)

```

Notas para o Formador:

- Este estudo de caso cobre preparação de dados, treino, avaliação e comparação de algoritmos.

Nota teórica: A comparação entre modelos permite avaliar a robustez e generalização, identificando o melhor compromisso entre desempenho e interpretabilidade.

- O dataset pode ser obtido de fontes abertas ou gerado sinteticamente.

Nota teórica: Datasets sintéticos são úteis para ensino, pois permitem controlar variáveis e introduzir cenários específicos.

- Incentivar a experimentação com diferentes algoritmos e hiperparâmetros.

Nota teórica: A otimização de hiperparâmetros pode melhorar significativamente o desempenho, especialmente em modelos complexos como random forest ou SVM.

- Discutir a escolha de métricas adequadas ao problema.

Nota teórica: A escolha da métrica deve refletir o objetivo do negócio. Por exemplo, em churn, minimizar falsos negativos pode ser mais importante do que maximizar a acurácia.